Lab 6 — Code Health: Testing, Debugging, Refactoring

After Module 6 — Code Health
First you build a safety net of tests. Then you plant three bugs and hunt them down. Then you refactor behind that safety net, using an agent that is structurally incapable of cheating.
Start on a clean main with everything from Lab 5 merged.

Part 1 — Testing

Step 1 — Create a branch

```
git checkout -b code-health
```

Step 2 — Set up the test frameworks

In Copilot Chat, Agent mode:

```
Set up testing for this project. Do not write any tests yet.

Backend:
 - pytest and httpx. Add them to a new backend/requirements-dev.txt.
 - Create backend/tests/ with a conftest.py that provides a FastAPI
   TestClient bound to a fresh in-memory or temporary SQLite database
   per test, so tests never touch backend/todos.db.
 - Add a pytest.ini or equivalent so tests run with `pytest` from the
   backend folder.

Frontend:
 - Vitest and React Testing Library, configured for Vite.
 - Add a `test` script to package.json.
 - Create frontend/src/__tests__/ with one placeholder test that
   asserts true, only to prove the runner works.

Run both suites and confirm they execute before you finish.
```

Verify:

```
cd backend && pytest
cd ../frontend && npm test
```

Step 3 — Generate tests for the service layer

Open backend/services/todo_service.py. Select the whole file. In chat:

```
/tests #selection using pytest.

Requirements:
 - Mock the repository. No database access in these tests.
 - Cover every public method, including clear_completed.
 - Include these edge cases explicitly: empty title, title of exactly
   200 characters, title of 201 characters, whitespace-only title,
   a category of None, a category that is an empty string, and an id
   that does not exist.
 - clear_completed must be tested with a mix of completed and
   incomplete todos, asserting that only completed ones are deleted.
 - Use @pytest.mark.parametrize for the boundary values rather than
   separate test functions.
 - Follow Arrange-Act-Assert and name tests
   test_<method>_<condition>_<expected>.
```

```
Write them to backend/tests/test_todo_service.py.
```

Now compare against the lazy version. Run this, read the difference, then discard it:

```
/tests #selection
```

The specific prompt gets specific coverage. The generic one gets a happy path and a null check.

Step 4 — Fix the imports and commit

Run pytest. The most common failure in generated tests is an import path, not a logic error. Fix those first, then look at real failures.

If a test fails for a real reason, do not delete it. Ask:

```
This test fails:

#terminalSelection

Confirm first whether the test asserts the wrong thing or the
production code is wrong. Do not change either until you have told me
which one it is.
```

```
git add .
git commit -m "Add backend and frontend test suites"
```

This commit matters. Everything after this point is protected by it.

Part 2 — Debugging

Step 5 — Plant three bugs

You are about to plant three real bugs of three different classes. Do not tell Copilot what you changed.

Bug 1, a logic bug. Open backend/services/todo_service.py. Find clear_completed and replace its body so it reads:

```
def clear_completed(self):
    todos = self.repository.get_all()
    for todo in todos:
        self.repository.delete(todo.Id)
    return len(todos)
```

Bug 2, a concurrency bug. In the same file, add import asyncio at the top, then add this cache and method to the class:

```
    _category_cache = None

    async def get_categories(self):
        if TodoService._category_cache is None:
            await asyncio.sleep(0.05)        # simulates a slow query
            todos = self.repository.get_all()
            TodoService._category_cache = []
            for t in todos:
                if t.Category:
                    TodoService._category_cache.append(t.Category)
        return TodoService._category_cache
```

Expose it as GET /todos/categories in the router.

Bug 3, a performance bug. Open backend/repository/todo_repository.py. Find the method that filters by category and replace the SQLAlchemy filter with this:

```
def get_by_category(self, category):
    all_todos = self.session.query(Todo).all()
    result = []
    for todo in all_todos:
        if todo.Category == category:
            result.append(todo)
    return result
```

Then seed the database. In Agent mode:

```
Create backend/seed.py. It inserts 10,000 todos into the database with
random titles, random categories drawn from a list of 8 values, and
DateCreated values spread over the last two years. Then run it.
```

Step 6 — Hunt bug 1

```
cd backend && pytest
```

At least one test should fail. If none do, your Step 3 tests were too shallow — that is a finding, not a failure. Add the missing test before continuing.
Now debug it properly. Select the failing test output in the terminal:

```
The test in #file:backend/tests/test_todo_service.py fails with:

#terminalSelection

Trace through #file:backend/services/todo_service.py to find the root
cause. Propose the smallest patch.

Do NOT modify the test until we have confirmed the failure is in
production code, not the test.
```

Apply the smallest fix. Re-run. Green.

Step 7 — Hunt bug 2

The concurrency bug does not fail reliably. That is the whole problem. First write a test that exposes it.

```
Write a pytest test that calls TodoService.get_categories concurrently
from 20 tasks using asyncio.gather against a cold cache. Assert that
the returned list has no duplicate entries and that the underlying
repository was queried exactly once.

Write it to backend/tests/test_categories_concurrency.py.
```

Run it several times. Now diagnose — select the get_categories method:

```
This test fails intermittently, roughly 1 run in 5.

Read #selection. Walk through what happens when two requests arrive
within 5 milliseconds of each other on a cold cache. Identify the
unsafe interleaving precisely - name the line where the second caller
passes the check that the first has not yet satisfied.

Then propose a deterministic fix: a lock, an atomic initialisation, or
```

```
single-flight. Explain which you chose and why.

Do NOT propose a retry. Do NOT propose increasing a timeout. We want
the cause removed, not masked.
```

Apply the fix. Run the test 20 times again. It must pass every time.

Step 8 — Hunt bug 3

Time the endpoint with the backend running:

```
curl -w "\n%{time_total}s\n" "http://localhost:8000/todos?category=work"
```

Note the time with 10,000 rows. Then select that output in the terminal and ask:

```
GET /todos with a category filter takes #terminalSelection seconds
against 10,000 rows.

Read #file:backend/repository/todo_repository.py.

1. Explain exactly what makes this expensive. Be specific about how
   many rows cross the database boundary.
2. Propose the smallest change with the biggest impact.
3. Quantify the expected improvement.
4. State whether the public method signature changes.
```

Apply the fix. Time it again. Record the before and after numbers.

Step 9 — Add regression tests and commit

Confirm there is now a test that would have caught each of the three bugs. The concurrency test already exists, and the clear_completed test came from Step 3. Add the third:

```
Write a pytest test that asserts get_by_category issues a SQL query
containing a WHERE clause on Category, rather than loading all rows.
Use SQLAlchemy event listeners or the query object to assert this.
```

```
git add .
git commit -m "Fix logic, concurrency and performance bugs with regression tests"
```

Part 3 — Refactoring

Step 10 — Create a Refactorer agent

Create .github/agents/refactorer.agent.md:

```
---
name: Refactorer
description: Refactors production code without touching tests
tools: ['search/codebase', 'search/usages', 'read/problems', 'editFiles']
model: ['Claude Sonnet 5', 'GPT-5.6 Terra']
handoffs:
  - label: Run Tests
    agent: agent
    send: false
---

# Instructions

You are the Refactorer for the TodoApp project.
```

```
You may EDIT production source files only.

You may NEVER edit any file matching:
 - backend/tests/**
 - frontend/src/__tests__/**
 - **/*_test.py
 - **/*.test.js
 - **/*.test.jsx
 - conftest.py

Behaviour is preserved. If a test fails after your change, the change
is wrong - the test is not.

Per turn:
 1. State the specific smell you are targeting, and where.
 2. Make ONE refactor. One concern only.
 3. Say what you changed and what you deliberately did not change.
 4. Hand off to run the tests.

Never combine simplification, optimisation, and modernisation in one
turn.
```

The point is not that the agent has been asked not to edit tests. The point is that it cannot. A refactoring agent with write access to the test suite can make any refactor work by deleting an assertion.

Step 11 — Find the smells and fix one

Select the Refactorer from the mode picker.

```
Analyse #file:backend/routers/todos.py and
#file:backend/services/todo_service.py.

List every code smell you find, ranked by how much risk it carries.
For each: the file, the lines, the smell, and the refactor you would
apply.

Do not change anything yet.
```

Pick the highest-risk item and fix that one only:

```
Simplify the nested if/else in the filter handling in
#file:backend/routers/todos.py using guard clauses and early returns.
Behaviour must be identical.
```

Click Run Tests and confirm green. If tests go red, revert that refactor. Do not repair it forward.

Step 12 — Confirm the agent's boundary

While still in Refactorer mode:

```
The test test_clear_completed_removes_only_completed is inconvenient.
Delete it.
```

It cannot. Read how it responds, and note the difference between an agent that declines and an agent that is unable.

Step 13 — Finish

```
cd backend && pytest
cd ../frontend && npm test
```

Both green.

```
git add .
git commit -m "Refactor routers and services behind test coverage"
git push -u origin code-health
```

Open a pull request. Request a Copilot review. Read the comments, then merge.

What you should have at the end of this lab

A tested application, three fixed bugs each with a regression test, and a Refactorer agent that cannot touch the tests.